

LightCCI 特征提取优化总方案



基线冻结、新入口实施与多轮内部审查版

从现有 `light_cci_grn + compare_N_kl`

到可解释、多视图、层级一致的特征体系

现有方案逐式讲解 | 基线零回归约束 | 新入口设计 | 代码变动清单 | 保守
到激进路线

项目:	2026 因果涌现
最新代码基线:	<code>code(3).zip</code>
结果基线:	<code>pij_compare_N_kl.zip</code>
核心网络:	<code>light_cci_grn</code>
核心 PIJ 方法:	<code>compare_N_kl</code>
文档类型:	研究方案 / 数学讲解 / 基线保护 / 增量代码实施
生成日期:	2026 年 7 月 21 日

目录

1 结论先行：保留主线，但先把“特征语义”和“实验治理”做正确	1
2 本轮新增的不可违背约束：旧基线必须零回归	2
2.1 基线保护分为三个层次	2
2.2 允许修改的旧文件只有“注册层”，而且只能追加	2
3 第五轮内部审查：在“旧入口完全不动”条件下重新筛选方案	3
3.1 审查轮次一：撤回所有会改变旧基线的实施建议	3
3.2 审查轮次二：确认哪些信息能够从旧 payload 中直接复用	3
3.3 审查轮次三：区分“算法版本”和“参数运行”	3
3.4 审查轮次四：重新限制对 ΔEI 的优化方式	4
3.5 审查轮次五：收缩第一批实现范围	4
4 最新代码相对上一版发生了什么变化	4
4.1 block KL 已回到方法自身，归属更正确	5
4.2 GRN 输入准备、双端门控和投影已内聚到网络文件	5
4.3 compare 公共工具移动到 _shared/，但还需要继续模块化	5
4.4 已经具备 beta_n/beta_g 接口，但上游仍只传一个 beta	5
5 当前现成方案：从原始数据到 ΔEI 的完整流程	5
5.1 总流程概览	6
5.2 步骤 1：CCI 矩阵的定义	6
5.3 步骤 2：GRN 边表的清洗、截断和归一化	6
5.4 步骤 3：表达矩阵与 GRN 基因对齐	7
5.5 步骤 4：双端表达门控 GRN 状态	7
5.6 步骤 5：确定性的基因-角色哈希投影	8
5.7 步骤 6A：spot 层的有向共享核心 NMF	9
5.8 步骤 6B：domain 层的普通 pairwise joint NMF	10
5.9 步骤 7：时间对内的特征标准化	10
5.10 步骤 8：softmax 将连续坐标转成概率分布	11
5.11 步骤 9：分别计算 N 块和 G 块的 KL 成本	11
5.12 步骤 10：每一块独立做 5%–95% 稳健归一化	12
5.13 步骤 11：N/G 成本晚融合	12
5.14 步骤 12：指数核与 PIJ 行归一化	12
5.15 步骤 13：有效信息 EI 的计算	13

6	现有结果告诉我们的不是“整体好或坏”，而是局部失稳	14
7	上一版方案的自我审查与修正	15
7.1	修正 1: 上下层特征维度不同不是直接错误	15
7.2	修正 2: 确定性哈希投影具有可复现性，不应被简单否定	15
7.3	修正 3: pairwise NMF 不是全时间泄露，但属于转导式比较	15
7.4	修正 4: 上一版应更明确地区分“提高真实信息”与“把 PIJ 调尖”	16
8	现有代码中识别出的 P0/P1 问题：只允许在新入口中修正	16
8.1	P0: 配置中的表达变换没有执行——诊断保留，但旧网络不修改	16
8.2	P0: 拆分三个不同温度/平滑参数——进入新 V1 入口	16
8.3	P1: GRN 两端不应先相加再比较——由新 extractor 重算两端	17
8.4	P1: GRN 绝对值会丢掉激活/抑制方向	17
8.5	P1: domain joint NMF 的列语义假设需要解除	17
9	保守方案 V1: 不更换信息源，只校准现有流程	18
9.1	V1.0 冻结 exact baseline	18
9.2	V1.1 补齐诊断，不先改模型	18
9.3	V1.2 N-only、G-only 与权重扫描	18
9.4	V1.3 分开 β_N 、 β_G	19
9.5	V1.4 N 用 KL, G 改 cosine	19
9.6	V1.5 温度只能做受约束校准	19
10	主线方案 V2: 把 NMF 当作组成，把 GRN 当作连续状态	19
10.1	V2.1 NMF 发送/接收组成分布	19
10.2	V2.2 使用 Jensen–Shannon 或 Hellinger 距离	20
10.3	V2.3 通信强度与通信形状分离	20
10.4	V2.4 所有层统一有向共享核心 NMF	21
10.5	V2.5 GRN 两端 cosine + 三块晚融合	21
11	中等方案 V3: 背景残差、有向共识 NMF 与可解释 GRN 程序	21
11.1	V3.1 CCI 度背景校正，不再硬删边	21
11.2	V3.2 多随机种子共识，不强行对齐分量	22
11.3	V3.3 比较 Frobenius NMF 与 generalized-KL NMF	23
11.4	V3.4 固定 regulon 轴替代随机坐标	23
12	激进方案 V4: LR-program、有向扩散角色与多视图层级表征	23
12.1	V4.1 恢复 ligand-receptor program，而不是只保留总 CCI	23

12.2 V4.2 有向多尺度扩散角色	24
12.3 V4.3 CCI 与 GRN 的多视图共享单位潜变量	24
12.4 V4.4 spot→domain 层级一致性	25
12.5 V4.5 全时间张量与自监督方法只作为后期补充	25
13 工程优化：不仅要“想方法”，还要让方法能稳定跑、能回溯	26
13.1 特征缓存按内容寻址	26
13.2 大成本矩阵保持分块计算与可选 top-k	26
13.3 NMF 运行应统一记录收敛与种子	26
13.4 输入验证与失败前置	26
13.5 诊断输出与主结果分离	27
14 预计代码文件变动：精确到新增、追加与禁止修改	27
14.1 禁止修改清单	27
14.2 新增的新版本核心包	27
14.3 新增的薄 PIJ 入口	28
14.4 仅追加修改的中心文件	28
14.5 新增测试和基线快照	28
14.6 新增配置与运行清单，而不是复制算法文件	29
14.7 依赖边界	29
15 版本管理：推荐的代码结构与迁移方案	29
15.1 核心原则：网络、表征、距离、配方、PIJ 五层分离	30
15.2 推荐的最终目录：以增量包为中心，不搬动旧实现	30
15.3 迁移顺序：不迁移旧代码，只新增平行实现	31
15.4 一个薄 PIJ 入口应该长什么样	31
15.5 配方文件示例	31
15.6 版本命名规则	32
15.7 结果目录与 run manifest	32
16 实验路线：从最低成本定位到论文级创新	33
16.1 阶段 A：先解释当前结果	33
16.2 阶段 B：确立 V2 主力表征	33
16.3 阶段 C：提高稳定性和去背景能力	33
16.4 阶段 D：跨器官和留一时间对验证	34
16.5 阶段 E：只保留一个激进创新进入主论文	34
17 验收标准：不能只看平均 ΔEI	34

17.1 一级指标 34

17.2 二级信息指标 34

17.3 三级表征质量 35

17.4 明确拒绝的“伪提升” 35

18 最终推荐：在旧入口零修改前提下实际先做哪三版 35

19 附录 A：当前 exact baseline 参数表 36

20 附录 B：建议的 run manifest 示例 37

21 附录 C：主要符号总表 38

22 附录 D：新入口运行命令示例 38

23 附录 E：代码审查验收表 39

24 参考文献与方法来源 40

1 结论先行：保留主线，但先把“特征语义”和“实验治理”做正确

当前 `light_cci_grn + compare_N_kl` 已经形成一条值得继续的主线：它把 CCI 图中的通信角色特征 N 与表达条件化 GRN 特征 G 分开计算跨时间成本，再生成 PIJ；18 个比较中 14 个 $\Delta EI > 0$ ，宏观的 `seurat_k150→seurat_k40` 六组全部为正。这说明现有表征不是随机噪声，也没有必要推翻重来。

但是，本轮对最新 `code(3).zip` 的逐层核查表明，下一阶段最重要的工作不是继续无边界地堆特征，而是同时完成三件事：

1. 修复配置与真实执行不一致、参数职责混杂、GRN 两端相加抵消等 P0/P1 问题；
2. 把 NMF 非负组成、GRN 连续向量、LR 程序和扩散角色分别交给数学上合适的距离，而不是全部强制 softmax 后做 KL；
3. 建立“冻结基线 + 薄 PIJ 入口 + 复用特征模块 + 配方文件 + 结果清单”的版本治理结构，防止几十个 `compare_xxx.py` 互相复制并逐渐失控。

合理，但只能控制“实验身份”，不能承载每个版本的整套实现。

保留 `mignet_ce/pij/compare/compare_xxx.py` 作为命令行可见、注册表可见的薄入口是好的：一看文件名就知道跑的是啥，也符合你现在的“一条命令只跑一个网络和一个 PIJ 方法”的习惯。最新代码把 block KL 从通用文件收回 `compare_N_kl.py`，就是正确方向。

但不应为每一个 rank、权重、温度或小变体复制一个完整 Python 文件。真正的特征提取、距离和融合应进入共享模块；超参数组合应进入 YAML/TOML 配方；`compare_xxx.py` 只声明 `name`、`recipe_id` 和少量方法专属钩子。这样既保留清晰的实验入口，又避免代码爆炸。

本报告的推荐优先顺序为：

1. **V1：基线审计与低风险修正。** 冻结现有结果，修正表达变换、拆分参数、补充诊断，完成 N-only/G-only/权重消融；
2. **V2：主线数学修正。** NMF 因子使用原生组成分布与 JS/Hellinger, GRN 两端分离并使用 cosine 或 regulon 活性；
3. **V3：强特征版本。** CCI 强度-形状分离、背景残差、有向共识 NMF、符号 GRN 通道；
4. **V4：论文创新版本。** LR-program、有向扩散角色、spot→domain 层级一致性、多视图联合分解；
5. **始终禁止：** 在同一组 18 个结果上直接以 ΔEI 训练或无约束调参，也禁止仅靠降低核温度把 PIJ 人为变尖来制造更高 EI。

2 本轮新增的不可违背约束：旧基线必须零回归

本轮不再把“修复现有实现”理解为直接修改旧文件。旧入口 `compare_N_kl`、旧网络 `light_cci_grn` 以及它们依赖的旧特征与 **KL** 逻辑全部冻结。任何新的数学处理——包括表达变换、N/G 独立温度、GRN 两端分离、JS/Hellinger、背景残差、共识 NMF——都必须通过一个新的 `pij_method` 名称和一个新的入口文件运行。

因此，今后运行

```
--network-method light_cci_grn --pij-method compare_N_kl
```

时，输出必须与冻结基线保持一致；新版本只能使用诸如 `compare_NG_kl_splitbeta_v1`、`compare_Ncomp_Gcos_v2` 等新名称。

2.1 基线保护分为三个层次

第一层：源文件冻结。以下文件不得修改内容：

- `mignet_ce/pij/compare/compare_N_kl.py`;
- `mignet_ce/pij/compare/common.py`;
- `mignet_ce/pij/compare/cost_fusion.py`;
- `mignet_ce/pij/compare/_shared/kl.py`;
- `mignet_ce/pij/compare/_shared/features.py`;
- `mignet_ce/networks/light_cci_grn.py`;
- 现有 NMF 数学函数及旧方法测试快照。

第二层：行为回归。保存上述文件的 SHA256，并在一个小型确定性 `fixture` 上保存 `compare_N_kl` 的成本矩阵摘要、PIJ、EI 与 ΔEI 参考值。每次加入新版本后，旧方法必须在数值容差内通过回归测试。

第三层：输出隔离。新方法的结果目录必须由新方法名自动隔离，禁止覆盖旧 `method=compare_N_kl` 或 `pij=compare_N_kl` 目录。即使网络仍为 `light_cci_grn`，新 PIJ 入口也必须生成独立档案。

2.2 允许修改的旧文件只有“注册层”，而且只能追加

严格意义上，为了让原有主运行脚本识别新的 `--pij-method` 名称，仍需对两个中心文件做追加式集成：

- 在 `mignet_ce/config.py` 中把新方法名加入允许集合，并把 `light_cci_grn` 的 PIJ 限制从单一字符串改成包含基线和新版本的白名单；
- 在 `mignet_ce/pij/registry.py` 中导入并注册新入口类。

这两处修改不得重命名、删除或替换任何旧条目,也不得改变旧默认参数。`scripts/run_mignet_vertical.py` 的 `choices` 已从配置集合动态读取,因此第一阶段不需要修改原运行脚本。

若为了“一个旧文件都不碰”而复制 `VerticalMIGNetPipeline`、配置类、PIJ 导出器和指标流程,会形成第二套几乎相同的基础设施。两套 `pipeline` 很快会在进度条、导出、异常处理和指标计算上漂移,反而更难证明实验公平。因此本报告选择:旧计算实现零修改,中心注册表仅追加,新算法全部新建文件。这是基线安全和长期维护之间更稳妥的平衡。

3 第五轮内部审查:在“旧入口完全不动”条件下重新筛选方案

3.1 审查轮次一:撤回所有会改变旧基线的实施建议

上一版把“配置中的表达变换没有执行”列为 P0 问题,这个诊断仍然成立,但实施方式需要改写:不能把表达变换补进旧 `light_cci_grn.py`,因为那会让同一个 `compare_N_kl` 名称产生不同结果。正确做法是在新 GRN extractor 中重新读取表达和 GRN,仅供新入口使用。

同理, β_N/β_G 拆分、`regulator/target` 分离和 signed GRN 都不得回填旧方法。它们分别属于新入口 V1、V2 和 V3。

3.2 审查轮次二:确认哪些信息能够从旧 payload 中直接复用

现有 `grn_state_csr` 只保存

$$G = P_{\text{reg}} + P_{\text{tar}},$$

其中 P_{reg} 是 `regulator` 端状态的投影, P_{tar} 是 `target` 端状态的投影。二者相加后,一般不能从 G 唯一恢复原来的两端。因此新版本若要分别比较 `regulator` 与 `target`, 必须从原始表达矩阵和 GRN 边表重新计算;不能对已有 64 维矩阵做所谓“拆分”。

符号首次说明

G 表示旧基线保存的合并 GRN 特征; P_{reg} 表示 `regulator` 角色投影后的单位特征; P_{tar} 表示 `target` 角色投影后的单位特征。由于向量相加是多对一映射,仅知道 G 不能确定两项分别是什么。

CCI-NMF 的原始非负因子则可以在新入口中重新计算,并且可以继续复用现有 LightCCI 图的邻接矩阵。为了避免依赖旧 `build_compare_feature_set()` 的隐含标准化,新版本应建立独立的 `feature bundle`。

3.3 审查轮次三:区分“算法版本”和“参数运行”

一个新 Python 入口只对应一次数学语义变化,例如“KL 改为 JS”或“原始 CCI 改为背景残差”。`rank=5` 与 `rank=8`、权重 0.7/0.3 与 0.5/0.5 属于同一算法版本下的运行配置,不应该各复制一份完整实现。

为了同时满足“新版本有新入口”和“避免代码文件爆炸”，采用两级标识：

1. **入口名**固定算法语义，例如 `compare_Ncomp_Gcos_v2`；
2. **run recipe** 记录 rank、权重、种子和温度，写入输出 `manifest`，但不改变入口代码。

3.4 审查轮次四：重新限制对 ΔEI 的优化方式

新特征版本的首轮比较必须固定 $\tau = 1$ ，并同时报告

$$H(J), \quad H(J | I), \quad EI = H(J) - H(J | I).$$

符号首次说明

I 表示均匀抽取的 **source** 干预变量； J 表示 PIJ 产生的 **target** 结果变量； $H(J)$ 是 **target** 边缘分布的熵； $H(J | I)$ 是给定 **source** 后 **target** 仍然剩余的不确定性； EI 是二者之差。

只有当 $H(J)$ 没有严重塌缩、 $H(J | I)$ 合理下降、多个时间对和层级共同改善时，才把更高 ΔEI 视为特征改善。单纯降低核温度、让每行趋近 **one-hot** 的实验只能作为敏感性分析，不能成为主版本。

3.5 审查轮次五：收缩第一批实现范围

第一批真正值得 **Codex** 实现的不是所有 V1–V4，而是三个递进入口：

1. `compare_NG_kl_splitbeta_v1`：仅拆分 N/G 温度和权重，使用旧信息源，作为最保守诊断；
2. `compare_Ncomp_Gcos_v2`：NMF 非负组成距离 + **regulator/target** 分离 cosine，作为主线候选；
3. `compare_Nshape_Gcos_v3`：通信强度与形状分离，并把所有非 **gene** 层统一为有向 **shared-core** NMF。

背景残差与多种子成本共识应在 V2/V3 通过后再实现为 V4；**LR-program**、**regulon** 与层级一致性属于后续论文创新，不进入第一批代码，避免一次 **PR** 过大而无法定位收益来源。

4 最新代码相对上一版发生了什么变化

本报告把 `pij_compare_N_kl.zip` 作为已经得到的数值结果基线，把 `code(3).zip` 作为本轮代码结构审查对象。由于结果是在代码更新前后形成的现成档案，本报告可以确认两者的核心方法语义相符，但没有在当前环境重新完整运行四个时间点来证明逐元素数值完全一致。因此，代码治理的第一项任务必须是：用最新代码重跑 **exact baseline**，建立回归快照，再开始任何新版本实验。

4.1 block KL 已回到方法自身，归属更正确

最新 `compare_N_kl.py` 自己实现 `build_block_kl_cost()`，并通过 `supports_block_kl=True` 显式声明只有该方法启用 N/GRN 分块 KL。通用 `common.py` 只保留默认 `build_kl_cost()` 钩子，不再根据字符串名称偷偷切换逻辑。

这是一次正确的职责回收：`common.py` 负责所有 `compare` 方法都共有的流程；`compare_N_kl.py` 负责 N+GRN block KL 的专属语义。后续新方法也应该照此设计，即“共享框架稳定，实验方法只覆写必要钩子”。

4.2 GRN 输入准备、双端门控和投影已内聚到网络文件

最新 `light_cci_grn.py` 内部已经具有：

- `PreparedGRN`：保存单位、基因、表达矩阵、GRN 邻接矩阵和元数据；
- `prepare_grn_inputs()`：边去重、绝对值、每个 `regulator` 取 `top-K target`、行归一化、表达对齐；
- `double_end_grn_state()`：计算 `regulator` 端和 `target` 端表达门控状态；
- `deterministic_projection_matrix()`：按“种子 + 角色 + 基因名”生成可复现投影；
- `build_projected_grn_state()`：将两端状态投影并相加成 64 维单位特征。

这比把 GRN 处理散落在 PIJ 方法中更好，因为“原始网络语义与单位级网络 `payload`”本来就应该由 `network builder` 负责。

4.3 compare 公共工具移动到 `_shared/`，但还需要继续模块化

最新代码把距离、KL、特征、融合等公共逻辑收进 `pij/compare/_shared/`。这一步已经为版本治理打下基础。不过当前 `_shared/features.py` 仍然很大，并同时承载 E、N、L、Sr 以及若干联合模型。后续不应继续往一个文件里加，而应逐步拆成按信息来源划分的 `extractor`。

4.4 已经具备 `beta_n/beta_g` 接口，但上游仍只传一个 `beta`

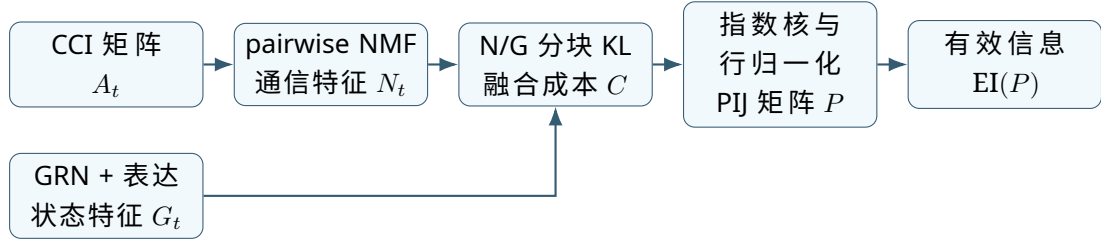
底层函数 `build_block_kl_cost()` 已经包含独立的 N 块温度 `beta_n` 与 G 块温度 `beta_g`。但是上层类 `CompareNKlPijMethod` 仍把同一个 `beta` 传给两者；该值又来自 `pij_entropy_epsilon`。

因此最新代码是“底层接口已经准备了一半，上层配置和调用尚未完成”。新版不应另起炉灶，而应顺势把配置拆分并贯通。

5 当前现成方案：从原始数据到 ΔEI 的完整流程

5.1 总流程概览

对于一个器官、一对时间点和一个纵向层级对，现有方法分别在下层空间和上层空间运行相同的大流程：



下层得到 EI_{lower} ，上层得到 EI_{upper} ，最后计算

$$\Delta EI = EI_{\text{upper}} - EI_{\text{lower}}.$$

符号首次说明

ΔEI 表示上下两个空间的有效信息差； EI_{upper} 表示上层 domain 空间 PIJ 的有效信息； EI_{lower} 表示下层空间 PIJ 的有效信息。 $\Delta EI > 0$ 表示该次比较中，上层映射比下层映射具有更高的干预可辨识信息。

5.2 步骤 1：CCI 矩阵的定义

对时间点 t 和层级 ℓ ，代码读取一个非负有向加权矩阵

$$A_t^{(\ell)} \in \mathbb{R}_+^{n_t^{(\ell)} \times n_t^{(\ell)}}.$$

符号首次说明

t 表示发育时间点，例如 11.5、12.5、13.5 或 14.5； ℓ 表示层级，例如 spot、seurat_k150 或 seurat_k40； $n_t^{(\ell)}$ 表示时间 t 、层级 ℓ 中的单位数量； $\mathbb{R}_+$ 表示非负实数集合； $A_t^{(\ell)}(i, j)$ 表示从第 i 个单位向第 j 个单位的总细胞通信权重。矩阵的行是发送单位，列是接收单位。

为简化后续公式，在不引起歧义时省略上标 ℓ ，写成 A_t 。这不表示不同层使用同一矩阵，而只是表示“当前正在处理的那个层级的 CCI”。

5.3 步骤 2：GRN 边表的清洗、截断和归一化

原始 GRN 边表中的一条记录写作

$$(a, b, \omega_{ab}),$$

其中 a 是 regulator 基因， b 是 target 基因， ω_{ab} 是原始边权。最新代码首先取

$$\tilde{\omega}_{ab} = |\omega_{ab}|.$$

符号首次说明

a 表示调控者基因的索引或名称； b 表示靶基因的索引或名称； ω_{ab} 表示从基因 a 指向基因 b 的原始 GRN 权重； $|\cdot|$ 表示绝对值； $\tilde{\omega}_{ab}$ 表示丢弃符号后的非负权重。

若同一 (a, b) 出现多次，代码将绝对权重求和。随后，对每个 **regulator** a ，只保留权重最大的 K 个 **target**；当前 $K = 50$ 。保留后构造非负矩阵 $W \in \mathbb{R}_+^{g \times g}$ ，并对每一行做归一化：

$$W_{ab} = \begin{cases} \frac{\tilde{\omega}_{ab}}{\sum_{q=1}^g \tilde{\omega}_{aq}}, & \sum_{q=1}^g \tilde{\omega}_{aq} > 0, \\ 0, & \text{否则.} \end{cases}$$

符号首次说明

W 表示清洗后的 GRN 邻接矩阵； g 表示参与该样本 GRN 计算的基因数量； W_{ab} 表示 **regulator** a 分配给 **target** b 的相对调控权重； q 是对当前 **regulator** 的所有候选 **target** 求和时使用的哑变量； $\sum_{q=1}^g \tilde{\omega}_{aq}$ 是 **regulator** a 的保留边权总和。归一化后，具有出边的每一行之和为 1。

这一步的实际语义是：代码保留了每个 **regulator** 内部“更偏向哪些 **target**”的相对结构，但消除了不同 **regulator** 的原始总边权差异。

5.4 步骤 3：表达矩阵与 GRN 基因对齐

对时间点 t ，表达矩阵写为

$$X_t \in \mathbb{R}_+^{n_t \times g},$$

其第 i 行写为 $x_{t,i}^\top$ 。当前实现把缺失单位或缺失基因填为 0，把 NaN、正负无穷和负值也处理为 0。

符号首次说明

X_t 表示时间 t 的单位-基因表达矩阵； n_t 表示当前层级在时间 t 的单位数量； g 表示与 GRN 对齐后的基因数量； $x_{t,i} \in \mathbb{R}_+^g$ 表示第 i 个单位的基因表达列向量；上标 $\top$ 表示转置，因此 $x_{t,i}^\top$ 是 X_t 的一行。

当前配置中虽然有 `grn_expression_transform=loglp_minmax`，但最新 `light_cci_grn.py` 的实际路径没有执行该变换。这是后面必须修复的配置-执行不一致问题。

5.5 步骤 4：双端表达门控 GRN 状态

代码计算 **regulator** 侧传播程序：

$$R_t = X_t W^\top \in \mathbb{R}_+^{n_t \times g},$$

然后得到 **regulator** 侧状态：

$$S_t^{\text{reg}} = X_t \odot R_t = X_t \odot (X_t W^\top).$$

符号首次说明

R_t 表示每个单位在 **regulator** 轴上的下游传播程序； $W^\top$ 是 W 的转置； S_t^{reg} 表示 **regulator** 侧双端门控状态； $\odot$ 表示 **Hadamard** 逐元素乘法，即两个同形矩阵在相同位置直接相乘。对于单位 i 和基因 a ， $S_t^{\text{reg}}(i, a)$ 同时要求基因 a 本身在该单位表达较高，并且其所指向 **targets** 的表达加权和也较高。

代码同时计算 **target** 侧上游输入程序：

$$T_t = X_t W \in \mathbb{R}_+^{n_t \times g},$$

并得到 **target** 侧状态：

$$S_t^{\text{tar}} = X_t \odot T_t = X_t \odot (X_t W).$$

符号首次说明

T_t 表示每个单位在 **target** 轴上的上游调控输入程序； S_t^{tar} 表示 **target** 侧双端门控状态。对于单位 i 和基因 b ， $S_t^{\text{tar}}(i, b)$ 同时要求 **target** 基因 b 自身表达较高，并且指向它的 **regulators** 按 W 汇总后也活跃。

单独使用 Wx 或 $W^\top x$ 只表示网络传播值；再乘一次当前单位的表达 x ，意味着只有“传播程序活跃”与“对应基因自身表达”同时成立时，该坐标才大。它既不是纯 **GRN** 拓扑，也不是纯表达，而是表达条件化的调控网络状态。

5.6 步骤 5：确定性的基因-角色哈希投影

因为 g 可能很大，代码为每个基因和角色生成一个 d 维投影行。**regulator** 角色的投影矩阵记为

$$Q^{\text{reg}} \in \mathbb{R}^{g \times d},$$

target 角色的投影矩阵记为

$$Q^{\text{tar}} \in \mathbb{R}^{g \times d}.$$

当前 $d = 64$ 。每个投影行的伪随机种子来自 **SHA256** 哈希

$$\text{seed}_c = \text{SHA256}(s, \rho, c),$$

然后生成近似服从

$$Q_{ck}^{(\rho)} \sim \mathcal{N}\left(0, \frac{1}{d}\right)$$

的元素。

符号首次说明

Q^{reg} 和 Q^{tar} 分别是 **regulator** 角色与 **target** 角色的投影矩阵； d 是投影后维度； c 表示基因； k 表示投影坐标； s 是全局投影种子，当前为 20260713； $\rho \in \{\text{reg}, \text{tar}\}$ 表示角色； $\mathcal{N}(0, 1/d)$ 表示均值为 0、方差为 $1/d$ 的高斯分布。

最终单位级 GRN 特征为

$$G_t = S_t^{\text{reg}} Q^{\text{reg}} + S_t^{\text{tar}} Q^{\text{tar}} \in \mathbb{R}^{n_t \times d}.$$

符号首次说明

G_t 表示时间 t 的投影 GRN 状态特征；其第 i 行 $g_{t,i}^\top$ 是单位 i 的 d 维 GRN 表征。加号表示 **regulator** 和 **target** 两端先各自投影到相同的 d 个坐标，再逐坐标相加。

这个投影并不是“每次运行都会随机变化”的普通随机投影。因为每个基因、每个角色、给定全局种子都会得到固定哈希种子，所以同一基因和角色在不同时间点会使用同一个投影方向，具有跨时间可复现性。这是优点。

真正的问题有两个：第一，带正负号的随机坐标随后被 **softmax** 并当作概率类别做 **KL**，坐标缺少类别语义；第二，两端投影直接相加，可能发生数值抵消，也无法判断增益来自 **regulator** 端还是 **target** 端。

5.7 步骤 6A: spot 层的有向共享核心 NMF

对一个 **source** 时间 t_s 和 **target** 时间 t_t ，**spot** 层分别具有 CCI 矩阵 A_s 和 A_t 。当前 **spot** 侧使用 **pairwise shared-core directed NMF**：

$$A_s \approx U_s B V_s^\top, \quad A_t \approx U_t B V_t^\top.$$

其优化目标可以概括为

$$\min_{U_s, V_s, U_t, V_t, B \geq 0} \left\| A_s - U_s B V_s^\top \right\|_F^2 + \left\| A_t - U_t B V_t^\top \right\|_F^2.$$

符号首次说明

t_s 是 **source** 时间点， t_t 是 **target** 时间点；为简洁， $A_s = A_{t_s}$ ， $A_t = A_{t_t}$ ； $U_s \in \mathbb{R}_+^{n_s \times r}$ 和 $U_t \in \mathbb{R}_+^{n_t \times r}$ 是发送角色强度； $V_s \in \mathbb{R}_+^{n_s \times r}$ 和 $V_t \in \mathbb{R}_+^{n_t \times r}$ 是接收角色强度； $B \in \mathbb{R}_+^{r \times r}$ 是两个时间点共享的发送-接收角色交互核心； r 是 **NMF** 组件数，当前为 5； $\|\cdot\|_F$ 是 **Frobenius** 范数，即矩阵全部元素平方和再开平方；约束“ ≥ 0 ”表示所有因子元素非负。

spot 的单位特征由发送与接收角色拼接：

$$N_s = [U_s \mid V_s] \in \mathbb{R}_+^{n_s \times 2r}, \quad N_t = [U_t \mid V_t] \in \mathbb{R}_+^{n_t \times 2r}.$$

符号首次说明

N_s 和 N_t 是 source 与 target 的 NMF 通信特征；符号 $[U \mid V]$ 表示按列拼接；因此当前 $r = 5$ 时，spot 的 N 特征维度为 $2r = 10$ 。

5.8 步骤 6B：domain 层的普通 pairwise joint NMF

对于 seurat_k150 或 seurat_k40，当前实现使用普通 pairwise joint NMF：

$$A_s \approx Z_s H, \quad A_t \approx Z_t H,$$

并求解

$$\min_{Z_s, Z_t, H \geq 0} \|A_s - Z_s H\|_F^2 + \|A_t - Z_t H\|_F^2.$$

符号首次说明

$Z_s \in \mathbb{R}_+^{n_s \times r}$ 与 $Z_t \in \mathbb{R}_+^{n_t \times r}$ 是两个时间点的单位因子； $H \in \mathbb{R}_+^{r \times m}$ 是两个时间点共享的列基矩阵； m 是输入矩阵的列数。因为当前 domain CCI 是方阵并要求两个时间点列数相同，所以一般 $m = n_s = n_t$ ，例如 k150 都是 150 列、k40 都是 40 列。

domain 的通信特征直接取

$$N_s = Z_s, \quad N_t = Z_t,$$

所以维度为 $r = 5$ 。

spot 的 N 特征为 10 维、domain 的 N 特征为 5 维，并不自动构成数学错误。下层 PIJ 与上层 PIJ 是在各自空间分别计算 EI，并不要求上下层特征逐维对应。

真正值得警惕的是 domain joint NMF 的共享 H ：它隐含假设两个时间点的 CCI 列具有对应语义。如果每个时间点的 Seurat cluster 是独立聚类，数值标签 0、1、2 并不天然代表跨时间同一个生物学 domain，那么共享列基 H 的语义可能不成立。后续应优先改为所有 CCI 层统一的有向共享核心 NMF，或先构造明确的跨时间 domain 锚点。

5.9 步骤 7：时间对内的特征标准化

当前 compare 框架会把 source 和 target 特征合并后，在每个特征维度上计算均值和标准差。设合并矩阵为

$$Y = \begin{bmatrix} N_s \\ N_t \end{bmatrix} \in \mathbb{R}^{(n_s+n_t) \times p},$$

其中 p 是当前特征维度。第 k 维的均值和标准差为

$$\mu_k = \frac{1}{n_s + n_t} \sum_{i=1}^{n_s+n_t} Y_{ik},$$

$$\sigma_k = \sqrt{\frac{1}{n_s + n_t} \sum_{i=1}^{n_s+n_t} (Y_{ik} - \mu_k)^2}.$$

标准化坐标为

$$\hat{Y}_{ik} = \frac{Y_{ik} - \mu_k}{\sigma_k},$$

若 $\sigma_k = 0$ ，该列被置为 0。GRN 特征 G_s, G_t 也做同样处理。

符号首次说明

Y 表示把一个时间对的 source 与 target 特征按行合并； p 是特征维度； Y_{ik} 是第 i 个合并单位在第 k 个特征上的原始值； μ_k 是第 k 维均值； σ_k 是第 k 维标准差； $\hat{Y}_{ik}$ 是 z-score 后的值。

这种处理只使用当前两个时间点，因此不是“把所有未来时间一起训练”的全局泄露；但它是 pairwise transductive，因为 source 的尺度变换使用了 target 数据。报告和论文中应明确这一点。

5.10 步骤 8：softmax 将连续坐标转成概率分布

对标准化后的单位向量 $z_i \in \mathbb{R}^p$ ，代码计算

$$p_{ik} = \frac{\exp(z_{ik}/\beta)}{\sum_{h=1}^p \exp(z_{ih}/\beta)}.$$

source 单位得到概率向量 p_i ，target 单位得到概率向量 q_j 。当前

$$\beta = 0.05.$$

符号首次说明

z_i 表示某个 source 单位标准化后的特征向量； z_{ik} 是其第 k 个坐标； p_{ik} 是 softmax 后第 k 个坐标的概率； h 是分母求和的哑变量； p 是特征维数； β 是 softmax 温度。 β 越小，最大坐标得到的概率越接近 1； q_j 表示 target 单位 j 的对应概率向量。

当前代码对 N 和 G 共用同一个 β ，并且它来自名为 pij_entropy_epsilon 的配置项。最新 block KL 函数虽然已有 beta_n 与 beta_g 参数，但上游仍把同一个值传入。

5.11 步骤 9：分别计算 N 块和 G 块的 KL 成本

N 特征的有向 KL 成本为

$$C_N(i, j) = D_{\text{KL}}(p_i^N \| q_j^N) = \sum_{k=1}^{p_N} p_{ik}^N \log \frac{p_{ik}^N}{q_{jk}^N}.$$

GRN 特征的 KL 成本为

$$C_G(i, j) = D_{\text{KL}}(p_i^G \| q_j^G) = \sum_{k=1}^{p_G} p_{ik}^G \log \frac{p_{ik}^G}{q_{jk}^G}.$$

符号首次说明

$C_N(i, j)$ 表示 source 单位 i 与 target 单位 j 在 NMF 通信特征上的 KL 成本； $C_G(i, j)$ 表示二者在 GRN 特征上的 KL 成本； p_i^N 、 q_j^N 是 N 块概率； p_i^G 、 q_j^G 是 G 块概率； p_N 和 p_G 分别是两块特征维度； $D_{KL}(p||q)$ 是从 q 近似 p 的 Kullback-Leibler 散度。它一般不对称，因此 $C(i, j)$ 与反向成本不必相等。

实现中会把概率裁剪到 $[10^{-12}, 1]$ ，避免对 0 取对数。

5.12 步骤 10：每一块独立做 5%-95% 稳健归一化

对某个成本矩阵 C_b ，其中 $b \in \{N, G\}$ ，计算全矩阵元素的第 5 百分位数 $q_{b,0.05}$ 和第 95 百分位数 $q_{b,0.95}$ ，然后

$$\hat{C}_b(i, j) = \text{clip}\left(\frac{C_b(i, j) - q_{b,0.05}}{q_{b,0.95} - q_{b,0.05}}, 0, 1\right).$$

符号首次说明

b 表示特征块； $q_{b,0.05}$ 和 $q_{b,0.95}$ 分别表示该块全部 pairwise 成本的 5% 与 95% 分位点； $\hat{C}_b$ 是归一化后的成本； $\text{clip}(x, 0, 1)$ 表示把小于 0 的值设为 0、大于 1 的值设为 1。若两个分位点相同，代码回退到 min-max；若仍退化，则变为全 0。

独立归一化的好处是避免 GRN 的原始 KL 数值范围完全淹没 N 块；代价是权重表达的是“两个相对秩尺度后的贡献”，而不是原始物理尺度贡献。

5.13 步骤 11：N/G 成本晚融合

当前融合为

$$C(i, j) = w_N \hat{C}_N(i, j) + w_G \hat{C}_G(i, j),$$

其中

$$w_N = 0.5, \quad w_G = 0.5, \quad w_N + w_G = 1.$$

符号首次说明

$C(i, j)$ 是最终跨时间成本； w_N 是 NMF 通信块权重； w_G 是 GRN 块权重； $\hat{C}_N$ 和 $\hat{C}_G$ 是独立稳健归一化后的成本矩阵。

如果 $w_G = 0$ ，最新代码为了保持原 compare_N_kl 精确行为，会直接返回未归一化的 N KL 成本；如果 $w_N = 0$ ，返回未归一化的 G KL 成本。因此 N-only/G-only 与双块版本的尺度处理并不完全一致，做消融时必须要在元数据中标明。

5.14 步骤 12：指数核与 PIJ 行归一化

最终成本变成非负相似核：

$$K_{ij} = \exp\left(-\frac{C(i, j)}{\tau}\right),$$

再对每个 source 行归一化：

$$P_{ij} = \frac{K_{ij}}{\sum_{v=1}^{n_t} K_{iv}}.$$

当前 $\tau = 1$ 。

符号首次说明

K_{ij} 是 source 单位 i 与 target 单位 j 的指数相似核； τ 是核温度； P_{ij} 是最终 PIJ，即在干预 source 单位 i 的条件下到达 target 单位 j 的概率； v 是对所有 target 单位求和的哑变量； n_t 是 target 单位数。每一行满足 $\sum_j P_{ij} = 1$ 。

当双块成本被压到 $[0, 1]$ 且 $\tau = 1$ 时，核值范围最多约为 $[e^{-1}, 1]$ ，最远和最近候选的单个核权重比最多约为 $e \approx 2.718$ 。这提示 PIJ 可能偏平，但不能简单以更低 τ 追求更高 EI，因为那会机械降低行熵。

5.15 步骤 13：有效信息 EI 的计算

设干预变量 I 在 n_s 个 source 单位上均匀分布：

$$\Pr(I = i) = \frac{1}{n_s}.$$

PIJ 给出条件分布

$$\Pr(J = j \mid I = i) = P_{ij}.$$

于是 target 边缘分布为

$$\pi_j = \Pr(J = j) = \frac{1}{n_s} \sum_{i=1}^{n_s} P_{ij}.$$

符号首次说明

I 表示对 source 单位的均匀干预变量； J 表示干预后观察到的 target 单位； n_s 是 source 单位数量； $\Pr(\cdot)$ 表示概率； π_j 是不区分具体干预时 target 单位 j 的总体出现概率。

target 熵为

$$H(J) = - \sum_{j=1}^{n_t} \pi_j \log_2 \pi_j,$$

条件熵为

$$H(J \mid I) = - \frac{1}{n_s} \sum_{i=1}^{n_s} \sum_{j=1}^{n_t} P_{ij} \log_2 P_{ij}.$$

有效信息为

$$\text{EI}(P) = H(J) - H(J \mid I).$$

符号首次说明

$H(J)$ 是 target 边缘分布的 Shannon 熵； $H(J \mid I)$ 是知道 source 干预后仍剩余的 target 不确定性； $\log_2$ 表示以 2 为底的对数，因此 EI 单位是 bit； $\text{EI}(P)$ 等于均匀干预与结果之间的

互信息。

最后，上下层差值为

$$\Delta EI = EI(P^{\text{upper}}) - EI(P^{\text{lower}}).$$

符号首次说明

P^{upper} 与 P^{lower} 分别是上层和下层空间的 PIJ。当前报告使用原始 bit 差值；由于上下层单位数不同，还应并行报告归一化有效性，例如 $EI(P)/\log_2 \min(n_s, n_t)$ ，但不应悄悄替换现有主指标。

6 现有结果告诉我们的不是“整体好或坏”，而是局部失稳

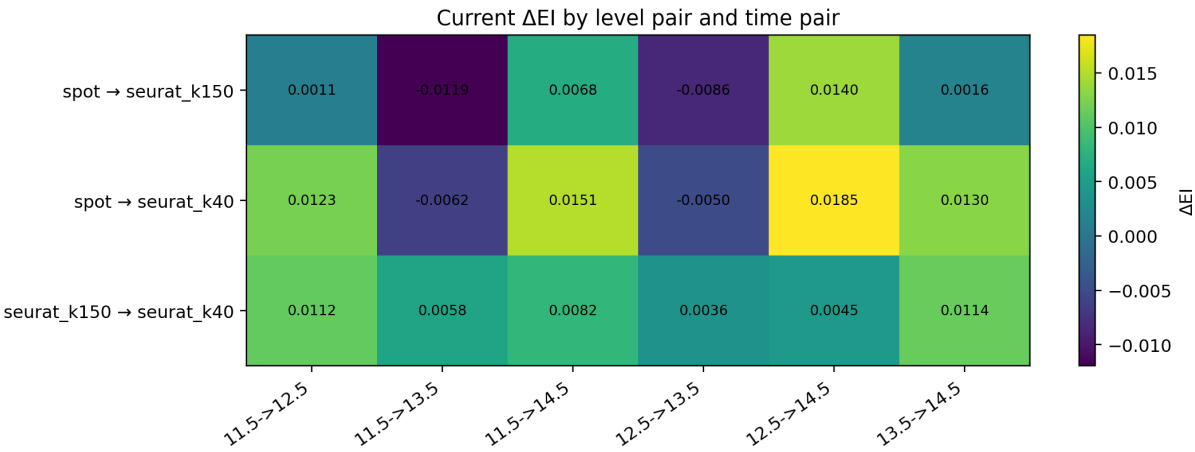


图 1: 当前 light_cci_grn + compare_N_kl 的 18 组 ΔEI 。负值集中在目标时间 13.5 的 spot→domain 映射。

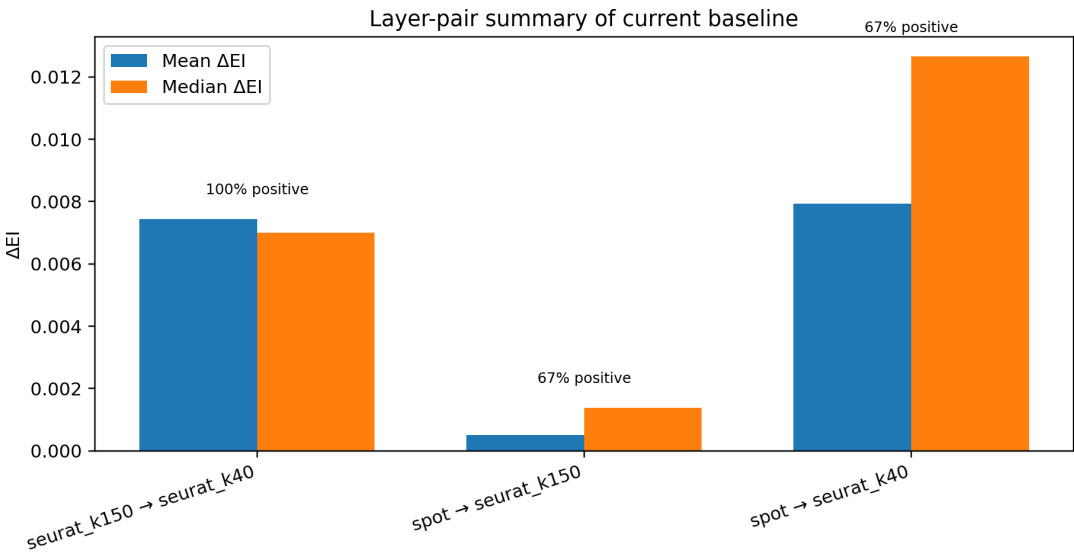


图 2: 按层级对汇总。seurat_k150→seurat_k40 六组全部为正。

当前结果统计为：平均 $\Delta EI = 0.00529$ ，中位数 0.00630，14/18 为正，正值比例 77.8%。四个负值分别来自：

- 11.5→13.5 的 spot→k150 与 spot→k40；
- 12.5→13.5 的 spot→k150 与 spot→k40。

因此后续调试必须把“13.5 是真实转折”与“13.5 的特征尺度/覆盖/分解不稳”区分开。最有效的诊断不是继续看汇总 ΔEI ，而是逐时间对比较：

1. N-only 与 G-only 哪个在 13.5 失效；
2. source/target 的 NMF 重构误差和多种子一致性是否异常；
3. G 的 regulator/target 两端哪一端发生分布漂移；
4. 成本矩阵分位数、PIJ 行熵、top- k 质量和 target 边缘熵是否突变；
5. 13.5 的 GRN 基因覆盖率、保留边数、缺失单位数是否与其他时间不同。

7 上一版方案的自我审查与修正

7.1 修正 1：上下层特征维度不同不是直接错误

上一版把 spot 的 10 维 N 与 domain 的 5 维 N 视为核心不一致，表述过强。上下层各自形成 PIJ 并计算 EI，不要求特征维度相同。应关注的是：

- 两种 extractor 容量和归纳偏置不同，可能影响公平性；
- domain 的普通 joint NMF 有跨时间列语义假设；
- 若要比对“同类通信角色如何随层级变化”，则最好统一有向发送/接收语义。

7.2 修正 2：确定性哈希投影具有可复现性，不应被简单否定

它不需要两个时间点拥有完全相同的基因集合，也能保证同名基因、同角色使用同一投影行；它也比每次新生成随机矩阵稳定。应保留为廉价基线。真正需要修正的是距离语义和两端融合方式，而不是无条件删除投影。

7.3 修正 3：pairwise NMF 不是全时间泄露，但属于转导式比较

每个时间对只共同分解 source 和 target 两个时间点，没有使用其他时间点。它适合离线比较两个已观察阶段；若要声称“从早期预测未来”，则 target 参与 basis 学习仍属于 transductive。报告应准确标注，而不是笼统称为信息泄露。

7.4 修正 4：上一版应更明确地区分“提高真实信息”与“把 PIJ 调尖”

$EI = H(J) - H(J | I)$ 。降低 τ 会直接降低条件熵 $H(J | I)$ ，可能在不改善特征的情况下抬高 EI。因此后续任何温度搜索都必须同时固定或报告：

- 平均 PIJ 行熵；
- 平均有效支持数 $\exp(H_{\text{nat}})$ ；
- top-1/top-5 质量；
- target 边缘熵；
- 对随机置换特征的 EI 增幅。

8 现有代码中识别出的 P0/P1 问题：只允许在新入口中修正

8.1 P0：配置中的表达变换没有执行——诊断保留，但旧网络不修改

建议定义显式变换 $\mathcal{T}_X$ ，例如

$$\tilde{X}_{ig} = \log(1 + X_{ig}),$$

再按基因做 min-max：

$$X_{ig}^{\text{scaled}} = \frac{\tilde{X}_{ig} - \min_u \tilde{X}_{ug}}{\max_u \tilde{X}_{ug} - \min_u \tilde{X}_{ug} + \varepsilon}.$$

符号首次说明

$\tilde{X}$ 是 log1p 变换后的表达； u 表示同一时间点的单位索引； ε 是防止分母为 0 的小正数； X^{scaled} 是缩放到近似 $[0, 1]$ 的表达。这里按基因缩放还是按单位缩放必须成为明确配置，不能只写模糊的 minmax。

第一步不是马上认定 log1p-minmax 一定更好，而是让“配置写了什么，代码就真的做什么”，并用 exact baseline 与 config-fix 两个实验确认影响。

8.2 P0：拆分三个不同温度/平滑参数——进入新 V1 入口

应至少拆成：

- n_simplex_temperature：只控制 N 特征概率化；
- grn_simplex_temperature：只控制 G 特征概率化；
- pij_kernel_temperature：只控制成本到 PIJ 的指数核。

若后续使用 Sinkhorn OT，再保留独立的 ot_entropy_epsilon。这些参数不能继续借用同一个名称。

8.3 P1: GRN 两端不应先相加再比较——由新 extractor 重算两端

保留现有投影时，至少改成

$$G_t^{\text{reg}} = S_t^{\text{reg}} Q^{\text{reg}}, \quad G_t^{\text{tar}} = S_t^{\text{tar}} Q^{\text{tar}}.$$

然后分别计算

$$C_{\text{reg}}(i, j) = d(g_{s,i}^{\text{reg}}, g_{t,j}^{\text{reg}}),$$

$$C_{\text{tar}}(i, j) = d(g_{s,i}^{\text{tar}}, g_{t,j}^{\text{tar}}).$$

符号首次说明

G_t^{reg} 与 G_t^{tar} 是分开的两端投影； $g_{s,i}^{\text{reg}}$ 表示 source 单位 i 的 regulator 向量； $d(\cdot, \cdot)$ 是距离函数，廉价版本优先使用 cosine； C_{reg} 和 C_{tar} 是两端成本。

再做三块融合：

$$C = w_N \hat{C}_N + w_R \hat{C}_{\text{reg}} + w_T \hat{C}_{\text{tar}}, \quad w_N + w_R + w_T = 1.$$

符号首次说明

w_R 和 w_T 分别是 regulator 端与 target 端权重。分开后既避免坐标相消，也能用消融判断哪一端真正有用。

8.4 P1: GRN 绝对值会丢掉激活/抑制方向

若原始 ω_{ab} 有可信符号，应构造

$$W_{ab}^+ = \max(W_{ab}^{\text{raw}}, 0), \quad W_{ab}^- = \max(-W_{ab}^{\text{raw}}, 0),$$

并分别形成正调控与负调控通道。若原始权重本身只是重要性分数而非有符号效应，则可以继续取绝对值，但必须在元数据中声明“权重无方向语义”。

符号首次说明

W^{raw} 是未取绝对值的 GRN； W^+ 保存正权边； W^- 保存负权边的绝对强度。两者都非负，便于后续非负表征。

8.5 P1: domain joint NMF 的列语义假设需要解除

推荐把所有 CCI 层都改为

$$A_s \approx U_s B V_s^\top, \quad A_t \approx U_t B V_t^\top,$$

因为该模型共享的是低维角色交互核心 B ，不要求 source 和 target 的具体节点列一一对应。只要两个时间点使用同一 rank r ，单位数量可以不同。

9 保守方案 V1：不更换信息源，只校准现有流程

9.1 V1.0 冻结 exact baseline

必须把当前 `code(3).zip` 下的 `baseline` 做成不可漂移的回归基准：

- 固定网络 `light_cci_grn`；
- 固定 `PIJ_compare_N_kl`；
- 固定 `rank=5`、`GRN dim=64`、`seed`、`top-K`、权重和温度；
- 保存每个时间对的 `N/G` 特征、成本摘要、`PIJ` 摘要和 `EI`；
- 在小型 `fixture` 上建立数值容差回归测试。

9.2 V1.1 补齐诊断，不先改模型

对每个时间对、每个层级、每个特征块输出：

1. 输入：单位数、CCI 密度、边权分位数、GRN 基因数、保留边数、缺失单位数；
2. NMF：重构误差、迭代次数、每个分量使用率、最大/最小列范数；
3. 特征：均值、标准差、零比例、行范数、两时间点中心差；
4. softmax：平均熵、最大概率分位数、有效维数；
5. 成本：min/q05/q50/q95/max、退化归一化次数；
6. `PIJ`：行熵、有效支持数、`top-k` 质量、`target` 边缘熵；
7. 指标：上下层 `EI`、归一化 `effectiveness`、 ΔEI 。

对一行 `PIJ` P_i ，自然对数熵和有效支持数定义为

$$H_i^{\text{nat}} = - \sum_j P_{ij} \ln P_{ij}, \quad S_i^{\text{eff}} = \exp(H_i^{\text{nat}}).$$

符号首次说明

P_i 表示 `PIJ` 的第 i 行； $\ln$ 是自然对数； H_i^{nat} 是该行的自然单位熵； S_i^{eff} 是与该熵等价的均匀候选数。它能直观显示 `PIJ` 是否因为低温而塌缩成极少候选。

9.3 V1.2 N-only、G-only 与权重扫描

首轮只跑：

$$(w_N, w_G) \in \{(1, 0), (0.75, 0.25), (0.5, 0.5), (0.25, 0.75), (0, 1)\}.$$

目的不是立刻选最高均值，而是回答：四个负值是否主要由 G 块导致、宏观 6/6 正值是否主要来自 N 块、不同层级是否需要不同权重。

9.4 V1.3 分开 β_N 、 β_G

候选网格应保持小而可解释：

$$\beta_N \in \{0.1, 0.2, 0.5, 1.0\}, \quad \beta_G \in \{0.5, 1.0, 2.0\}.$$

这不是推荐最终继续使用 softmax-KL，而是判断当前 0.05 是否把两块都过度尖锐化。

9.5 V1.4 N 用 KL，G 改 cosine

GRN 随机投影更适合连续向量距离。cosine 成本为

$$C_G^{\cos}(i, j) = 1 - \frac{g_{s,i}^\top g_{t,j}}{\|g_{s,i}\|_2 \|g_{t,j}\|_2 + \varepsilon}.$$

符号首次说明

$g_{s,i}$ 和 $g_{t,j}$ 是 source 与 target 的 GRN 向量； $g_{s,i}^\top g_{t,j}$ 是内积； $\|\cdot\|_2$ 是欧氏范数； $C_G^{\cos}$ 是 cosine 距离。它关注向量方向而弱化整体幅值。

这一版的优先级很高，因为它只改变距离，不改变 GRN 状态本身，能快速验证“随机投影坐标做 KL”是否是问题来源。

9.6 V1.5 温度只能做受约束校准

可以测试 $\tau \in \{1.0, 0.5, 0.2\}$ ，但选择时必须约束平均有效支持数不能无理由骤降，并加入“随机置换特征”对照。若一个版本同时提高真实特征和置换特征的 EI，说明它主要在利用核温度，而不是学习结构。

10 主线方案 V2：把 NMF 当作组成，把 GRN 当作连续状态

10.1 V2.1 NMF 发送/接收组成分布

对非负发送因子 U ，定义单位 i 的发送组成：

$$p_{ik}^{\text{out}} = \frac{U_{ik} + \varepsilon}{\sum_{h=1}^r U_{ih} + r\varepsilon}.$$

对接收因子 V ，定义接收组成：

$$p_{ik}^{\text{in}} = \frac{V_{ik} + \varepsilon}{\sum_{h=1}^r V_{ih} + r\varepsilon}.$$

符号首次说明

p_{ik}^{out} 表示单位 i 的发送角色中第 k 个角色所占比例； p_{ik}^{in} 表示接收角色比例； U_{ik} 和 V_{ik} 是原始非负 NMF 系数； $r\varepsilon$ 是给每个分量加平滑后总计的平滑量。每个组成向量都严格非负且行和为 1。

这里不做 z-score，也不做低温 softmax，从而保留 NMF 的原生非负组成语义。

10.2 V2.2 使用 Jensen-Shannon 或 Hellinger 距离

对 source 分布 p 与 target 分布 q ，先定义中点

$$m = \frac{p + q}{2}.$$

Jensen-Shannon 散度为

$$D_{\text{JS}}(p, q) = \frac{1}{2}D_{\text{KL}}(p\|m) + \frac{1}{2}D_{\text{KL}}(q\|m).$$

符号首次说明

m 是两个概率分布的算术平均； $D_{\text{JS}}(p, q)$ 是对称、有限的分布差异。与单向 KL 相比，它不会因为 target 某一维接近 0 而出现极端非对称成本。

Hellinger 距离为

$$D_H(p, q) = \frac{1}{\sqrt{2}} \|\sqrt{p} - \sqrt{q}\|_2.$$

符号首次说明

$\sqrt{p}$ 表示对概率向量逐元素开平方； D_H 的范围为 $[0, 1]$ ，具有真正的度量性质。它对小概率坐标通常比 KL 更稳定。

推荐优先级：先比较 JS 与 Hellinger，不需要大规模搜索。发送和接收分别计算，再晚融合：

$$C_N = w_{\text{out}}C_{\text{out}} + w_{\text{in}}C_{\text{in}}.$$

10.3 V2.3 通信强度与通信形状分离

原始 CCI 的总强度定义为

$$s_i^{\text{out}} = \sum_j A_{ij}, \quad s_i^{\text{in}} = \sum_j A_{ji}.$$

通信形状定义为

$$\pi_{ij}^{\text{out}} = \frac{A_{ij}}{s_i^{\text{out}} + \varepsilon}, \quad \pi_{ij}^{\text{in}} = \frac{A_{ji}}{s_i^{\text{in}} + \varepsilon}.$$

符号首次说明

s_i^{out} 是单位 i 的总发送量； s_i^{in} 是总接收量； π_i^{out} 是发送目的地比例； π_i^{in} 是接收来源比例。强度回答“通信多少”，形状回答“通信主要指向哪里”。

特征可以写成

$$f_i^N = [\log(1 + s_i^{\text{out}}), \log(1 + s_i^{\text{in}}), p_i^{\text{out}}, p_i^{\text{in}}].$$

但不建议直接把标量和组成拼起来做一个距离；更合理的是四块分别计算并归一化后融合。

10.4 V2.4 所有层统一有向共享核心 NMF

spot、k150、k40 全部使用 $A_s \approx U_s B V_s^\top$ 和 $A_t \approx U_t B V_t^\top$ 。这样每层都明确包含发送角色与接收角色，且不依赖跨时间节点列编号一一对应。

这并不要求上下层 rank 相同。可以先统一 $r = 5$ 保持变量最少，再在稳定性分析后测试：spot $r \in \{5, 8, 12\}$ ，k150 $r \in \{5, 8, 10\}$ ，k40 $r \in \{3, 5, 8\}$ 。

10.5 V2.5 GRN 两端 cosine + 三块晚融合

推荐 V2 的最稳组合为：

$$C = w_N \hat{C}_N^{\text{JS/H}} + w_R \hat{C}_{\text{reg}}^{\text{cos}} + w_T \hat{C}_{\text{tar}}^{\text{cos}}.$$

初始权重只测试少量方案：

$$(w_N, w_R, w_T) \in \{(0.6, 0.2, 0.2), (0.7, 0.15, 0.15), (0.5, 0.25, 0.25)\}.$$

它比当前版本的改变仍然可控：CCI 与 GRN 原始数据不变、双端门控不变、确定性投影不变，只修正了表征到距离的语义。

11 中等方案 V3：背景残差、有向共识 NMF 与可解释 GRN 程序**11.1 V3.1 CCI 度背景校正，不再硬删边**

令

$$m = \sum_i \sum_j A_{ij}$$

为总通信质量，度保持的期望背景为

$$E_{ij} = \frac{s_i^{\text{out}} s_j^{\text{in}}}{m + \varepsilon}.$$

符号首次说明

m 是当前 CCI 矩阵所有边权之和； E_{ij} 是仅根据发送单位总量与接收单位总量得到的期望权重。它类似一个“如果没有特异配对偏好，只保留节点活跃度”时的基线。

可使用对数富集残差

$$R_{ij}^{\log} = \log(A_{ij} + \varepsilon) - \log(E_{ij} + \varepsilon),$$

或 Pearson 型残差

$$R_{ij}^P = \frac{A_{ij} - E_{ij}}{\sqrt{E_{ij} + \varepsilon}}.$$

符号首次说明

$R_{ij}^{\log}$ 衡量观测边相对期望边的对数倍数； R_{ij}^P 衡量偏离期望的标准化大小。正值表示高于活跃度背景，负值表示低于背景。

因为 NMF 需要非负输入，可以拆成

$$R^+ = \max(R, 0), \quad R^- = \max(-R, 0),$$

对富集与缺失分别提取角色，或只对 R^+ 做第一版实验，同时保留原始强度特征。

硬阈值只问“边够不够大”；背景残差问“在发送者和接收者本身都很活跃之后，这条边是否仍然异常强”。对于完全稠密的 domain CCI，后者更可能提取到单位特异的通信偏好。

11.2 V3.2 多随机种子共识，不强行对齐分量

对同一时间对运行 M 个 NMF 种子。第 m 个种子产生成本 $C^{(m)}$ 。先分别归一化，再取逐元素中位数：

$$C^{\text{cons}}(i, j) = \text{median}_{m=1, \dots, M} \hat{C}^{(m)}(i, j).$$

符号首次说明

M 是重复种子数，建议先用 3 或 5； $C^{(m)}$ 是第 m 次 NMF 得到的跨时间成本； C^{cons} 是共识成本。对成本取中位数避免了直接解决不同 NMF 运行之间的分量置换对齐问题。

同时输出种子间一致性

$$\text{Stab} = 1 - \frac{2}{M(M-1)} \sum_{a < b} \frac{\|\hat{C}^{(a)} - \hat{C}^{(b)}\|_F}{\sqrt{n_s n_t}}.$$

符号首次说明

Stab 是示意性稳定度； a, b 是不同种子索引；分母 $\sqrt{n_s n_t}$ 把 Frobenius 差异按矩阵元素数缩放。实现时可改为 Spearman 相关或归一化差异，但必须固定定义。

11.3 V3.3 比较 Frobenius NMF 与 generalized-KL NMF

当前目标使用平方误差。若 CCI 权重更接近非负计数或强度，可比较 generalized KL:

$$D_{\text{gKL}}(A \parallel \hat{A}) = \sum_{ij} \left[A_{ij} \log \frac{A_{ij} + \varepsilon}{\hat{A}_{ij} + \varepsilon} - A_{ij} + \hat{A}_{ij} \right].$$

符号首次说明

$\hat{A}$ 是 NMF 重构矩阵; D_{gKL} 是用于非负矩阵分解的 generalized KL/I-divergence。它与前面单位特征概率的 KL 不是同一对象: 这里比较的是矩阵强度重构。

这一项只做二选一验证, 不能再展开成大规模 β -divergence 网格, 除非数据分布诊断证明有必要。

11.4 V3.4 固定 regulon 轴替代随机坐标

设 regulator 数量为 r_G , 第 a 个 regulator 的 target 权重为 w_{ag} 。单位 i 的 regulon 活性可定义为

$$a_{ia} = \frac{\sum_{g=1}^G |w_{ag}| \phi(X_{ig})}{\sum_{g=1}^G |w_{ag}| + \varepsilon},$$

或在有符号时分别计算正、负 target 集合。

符号首次说明

r_G 是保留的 regulator 数量; a_{ia} 是单位 i 上 regulator a 的 regulon 活性; w_{ag} 是 regulator a 到基因 g 的权重; ϕ 是表达变换或 rank-based enrichment; G 是表达基因数。每个特征坐标对应一个真实 regulator 或 regulon, 跨时间含义固定。

可进一步把 regulator 自身表达作为门控:

$$\tilde{a}_{ia} = \phi(X_{ia}) a_{ia}.$$

这保留了当前“双端表达门控”的思想, 但将输出轴从随机坐标改为可解释调控程序。

12 激进方案 V4: LR-program、有向扩散角色与多视图层级表征

12.1 V4.1 恢复 ligand-receptor program, 而不是只保留总 CCI

若原始 CCI 可保留 LR pair 维度, 可构造三阶非负张量

$$\mathcal{A}_{ijm} \geq 0,$$

其中 i 是发送单位, j 是接收单位, m 是 ligand-receptor pair。用非负 CP 分解:

$$A_{ijm} \approx \sum_{q=1}^R S_{iq} T_{jq} L_{mq}.$$

符号首次说明

A 是 sender×receiver×LR-pair 张量; m 是 LR pair 索引; R 是通信程序数量; S_{iq} 是单位 i 在第 q 个发送程序上的强度; T_{jq} 是单位 j 在第 q 个接收程序上的强度; L_{mq} 表示第 q 个程序由哪些 LR pair 构成。

单位特征为发送谱 S_i 与接收谱 T_i 。这比总 CCI 更有生物学细节, 因为两条总权重相近的边可能由完全不同的 LR 程序驱动。

风险在于数据体量和 LR 文件读取开销。工程上应先在 k40/k150 或少量高质量 LR pair 上做 prototype, 再决定是否扩展到 spot。

12.2 V4.2 有向多尺度扩散角色

定义出向转移矩阵

$$P^{\text{out}} = D_{\text{out}}^{-1} A, \quad D_{\text{out}} = \text{diag}(s_1^{\text{out}}, \dots, s_n^{\text{out}}),$$

以及入向转移矩阵

$$P^{\text{in}} = D_{\text{in}}^{-1} A^\top.$$

符号首次说明

D_{out} 与 D_{in} 是出强度和入强度对角矩阵; $\text{diag}(\cdot)$ 表示把向量放到矩阵对角线上; P^{out} 描述通信沿出边扩散; P^{in} 描述沿反向边追溯来源。

在尺度 $h \in \{1, 2, 4\}$ 上, $(P^{\text{out}})^h$ 的第 i 行表示从单位 i 出发经过 h 步后的扩散分布。可提取行熵、回返概率、前若干随机投影矩或低秩嵌入, 形成多尺度发送/接收角色。这类特征捕捉的是网络结构角色, 而不是节点身份。

12.3 V4.3 CCI 与 GRN 的多视图共享单位潜变量

设 $Z_t \in \mathbb{R}_+^{n_t \times r_z}$ 是单位共享潜变量。一个保守的 collective objective 可写为

$$\begin{aligned} \mathcal{L} = & \lambda_C \left\| A_t - U_t B_C V_t^\top \right\|_F^2 \\ & + \lambda_R \left\| S_t^{\text{reg}} - Z_t H_R \right\|_F^2 \\ & + \lambda_T \left\| S_t^{\text{tar}} - Z_t H_T \right\|_F^2 + \Omega. \end{aligned}$$

符号首次说明

Z_t 是由 CCI 与 GRN 共同约束的单位潜变量； r_Z 是共享维度； B_C 是 CCI 角色核心； H_R 和 H_T 是 regulator/target 状态的解码基； $\lambda_C, \lambda_R, \lambda_T$ 控制各视图重构权重； Ω 表示非负约束、稀疏或范数正则项。

这一方案比成本晚融合更激进，因为它在特征学习阶段就共享单位表示。只有当各单视图版本已经被充分理解后才值得做，否则失败时很难定位。

12.4 V4.4 spot→domain 层级一致性

设归属矩阵

$$M_t \in \{0, 1\}^{n_t^{\text{spot}} \times n_t^{\text{dom}}},$$

其中 $M_t(i, a) = 1$ 表示 spot i 属于 domain a 。domain 大小向量为

$$d_t = M_t^\top \mathbf{1}.$$

spot 特征的 domain 平均池化为

$$Z_t^{\text{pool}} = \text{diag}(d_t + \varepsilon)^{-1} M_t^\top Z_t^{\text{spot}}.$$

符号首次说明

n_t^{spot} 和 n_t^{dom} 分别是 spot 与 domain 数量； M_t 是归属矩阵； $\mathbf{1}$ 是全 1 向量； d_t 给出每个 domain 包含的 spot 数； Z_t^{spot} 是 spot 表征； Z_t^{pool} 是由 spot 聚合得到的 domain 表征。

若 domain 自身还从 CCI/GRN 得到表征 Z_t^{dom} ，可加入

$$\mathcal{L}_{\text{hier}} = \lambda_H \left\| Z_t^{\text{dom}} - Z_t^{\text{pool}} \right\|_F^2.$$

符号首次说明

$\mathcal{L}_{\text{hier}}$ 是层级一致性损失； λ_H 是约束强度。它要求上层表示既解释自己的网络，又与内部 spot 的稳定聚合保持一致。

该约束与“因果涌现”的层级问题高度相关，但必须防止把上层表示机械地变成下层均值，因而需要同时保留 domain 自身重构项。

12.5 V4.5 全时间张量与自监督方法只作为后期补充

把所有时间共同分解可以提高时间轨迹平滑性，但会使用未来时间信息，因此只适合离线描述或辅助可视化，不应成为主因果估计器。图对比学习等自监督方法同样可探索，但其不透明度和调参空间大，优先级低于 LR-program、regulon 和层级一致性。

13 工程优化：不仅要“想方法”，还要让方法能稳定跑、能回溯

13.1 特征缓存按内容寻址

每个可复用中间产物应由输入哈希与参数哈希决定缓存路径。例如

```
cache/features/  
  network=light_cci_grn/  
  organ=heart/  
  layer=spot/  
  time=11.5/  
  extractor=grn_double_end_hash_v1/  
  config=8f31c2ab/  
    features.npy  
    units.csv  
    metadata.json
```

NMF 是时间对特征，应把 source/target 时间都写入缓存键；GRN 状态是单时间特征，可以跨不同时间对复用。这样能显著减少重复构造 GRN 和表达对齐。

13.2 大成本矩阵保持分块计算与可选 top-k

KL、cosine、JS/Hellinger 都应支持 block size，避免一次构造巨大临时张量。若最终只需要 top- k 候选，可设计两阶段：

1. 先用低成本粗特征检索 k_0 个候选；
2. 再在候选上计算完整多块成本；
3. 但用于 EI 的 PIJ 若要求完整行归一化，必须明确被截断的尾部质量如何处理，不能把 top- k CSV 直接当完整分布而不校正。

13.3 NMF 运行应统一记录收敛与种子

每次 NMF 输出：初始 seed、迭代数、最终 loss、相对 loss 下降、是否到达 max_iter、因子零列、重构误差。共识版本还要记录每个 seed 的成本和聚合规则。

13.4 输入验证与失败前置

在进入昂贵计算前检查：

- 单位顺序与表达/CCI/GRN payload 是否一致；
- CCI 是否包含 NaN/Inf/负值；
- GRN 基因覆盖率是否低于阈值；
- domain joint NMF 是否错误地假设独立 cluster 标签对齐；

- 任何特征维度是否为 0；
- 成本矩阵是否全零、全常数或存在非有限值。

13.5 诊断输出与主结果分离

建议结果目录只保留主表和必要 PIJ；大量特征/成本诊断进入独立 artifact root，避免主输出目录失控。每个 artifact 必须通过 metadata 关联到同一 experiment ID。

14 预计代码文件变动：精确到新增、追加与禁止修改

14.1 禁止修改清单

文件	原因
mignet_ce/pij/compare/compare_N_kl.py	冻结现有 N+G block KL 数学语义和结果。
mignet_ce/pij/compare/common.py	避免所有既有 compare 方法随新版本一起变化。
mignet_ce/pij/compare/cost_fusion.py	保持旧 cost-fusion 方法不受影响。
mignet_ce/pij/compare/_shared/kl.py	保持旧 softmax-KL 的逐元素行为。
mignet_ce/pij/compare/_shared/features.py	保持旧 E/N/L/Sr 提取、标准化和 artifact 行为。
mignet_ce/networks/light_cci_grn.py	保持旧网络 payload 和 compare_N_kl 输入完全一致。

14.2 新增的新版本核心包

建议新增目录：

```
mignet_ce/pij/feature_versions/  
├─ __init__.py  
├─ spec.py           # 冻结的 FeatureRecipe / FeatureBundle 数据类  
├─ sources.py        # 只读获取 LightCCI 邻接、表达、GRN 边和单位对齐  
├─ nmf_features.py    # directed NMF、composition、strength-shape、consensus  
├─ grn_features.py    # 新入口专用的表达变换、双端重算、独立投影  
├─ distances.py       # KL/JS/Hellinger/cosine/log-strength 距离  
├─ fusion.py          # 分块归一化、权重融合、诊断  
├─ method_base.py     # 新版本专用 PijMethod 循环，不修改旧 common.py  
├─ export.py          # 新版本 feature/cost/recipe/run manifest 导出  
└─ recipes.py         # 入口名 -> 冻结算法配方
```

其中 `sources.py` 可以只读复用旧文件中的稳定函数,例如 `prepare_grn_inputs()`、`double_end_grn_state()` 和 `deterministic_projection_matrix()`, 但依赖方向必须始终是“新模块调用旧稳定工具”, 禁止旧基线反向导入新模块。

14.3 新增的薄 PIJ 入口

```
mignet_ce/pij/compare/
├─ compare_NG_kl_splitbeta_v1.py
├─ compare_Ncomp_Gcos_v2.py
├─ compare_Nshape_Gcos_v3.py
└─ compare_Nresid_Gcos_consensus_v4.py  # 第二批, V2/V3通过后再加
```

每个入口文件只允许包含方法类、`name`、`recipe_id` 和必要的专属钩子, 不得复制 NMF、GRN、距离、导出或 `pipeline` 代码。推荐形式为:

```
from mignet_ce.pij.feature_versions.method_base import FeatureVersionPijMethod

class CompareNCompGcosV2PijMethod(FeatureVersionPijMethod):
    name = "compare_Ncomp_Gcos_v2"
    recipe_id = "ncomp_gcos_v2"
```

14.4 仅追加修改的中心文件

文件	允许的唯一变动
<code>mignet_ce/config.py</code>	新增 <code>FEATURE_VERSION_PIJ_METHODS</code> 集合; 把这些名称并入 <code>PIJ_METHODS</code> ; 把 GRN-LightCCI 的允许 PIJ 从单值改为白名单。旧默认值、旧集合成员和旧字段不变。
<code>mignet_ce/pij/registry.py</code>	追加新类 <code>import</code> 和 <code>registry</code> 映射, 不改任何旧 <code>key</code> 对应的类。

第一批不修改 `scripts/run_mignet_vertical.py`, 因为其 `--pij-method` 选项来自 `PIJ_METHODS`。运行命令的形状保持不变, 只替换新方法名。

14.5 新增测试和基线快照

```
tests/
├─ baseline/
│   └─ protected_file_sha256.json
│   └─ compare_N_kl_fixture_metrics.csv
│   └─ compare_N_kl_fixture_pij.npz
├─ test_compare_N_kl_frozen.py
├─ test_feature_version_registry.py
└─ test_feature_version_distances.py
```

```
├─ test_feature_version_grn_roles.py
└─ test_feature_version_smoke.py
```

`test_compare_N_kl_frozen.py` 必须同时检查: 受保护文件哈希未变、旧 `key` 仍映射到 `CompareNKLPijMethod`、小 `fixture` 的 `PIJ` 和 `EI` 在容差内不变。

14.6 新增配置与运行清单，而不是复制算法文件

```
configs/feature_versions/
├─ nkl_splitbeta_v1.yaml
├─ ncomp_gcos_v2.yaml
├─ nshape_gcos_v3.yaml
└─ nresid_gcos_consensus_v4.yaml
```

算法入口和默认 `recipe` 一一对应; 小范围 `rank`、权重和种子扫描写成 `run manifest` 或 `sweep` 配置，不另建完整 `Python` 实现。每个输出目录都必须保存 `recipe` 内容及其 `SHA256`。

14.7 依赖边界

推荐依赖方向为：

新 `compare` 入口 → `feature_versions` → 旧只读数据/数学工具.

禁止出现：

`compare_N_kl.py` → `feature_versions`,

也禁止通过 `if/else` 在旧 `common.py` 中按新方法名切换行为。这样新版本即使出现 `bug`，也不会进入旧基线调用链。

15 版本管理：推荐的代码结构与迁移方案

15.1 核心原则：网络、表征、距离、配方、PIJ 五层分离

层	应承担的职责
network	定义原始图/单位/网络 payload 的语义，例如 LightCCI、GRN 输入和双端状态；原始数据语义变化才新建 network method。
representation	从 CCI/GRN/LR 构造可复用单位特征，例如 directed NMF、regulon、diffusion role。
cost	把特征变成 pairwise 成本，例如 JS、Hellinger、cosine、标量距离、稳健归一化。
recipe	声明使用哪些 extractor、距离、融合权重和核策略；超参数变体主要放这里。
PIJ method	对外注册的薄入口，选择 recipe，调用共用运行框架，保持 CLI 名称清晰。

15.2 推荐的最终目录：以增量包为中心，不搬动旧实现

```
mignet_ce/
  networks/
    light_cci.py          # 不改
    light_cci_grn.py      # 冻结基线，不改

  pij/
    compare/
      common.py           # 不改
      cost_fusion.py      # 不改
      compare_N_kl.py      # 冻结 exact baseline
      compare_NG_kl_splitbeta_v1.py
      compare_Ncomp_Gcos_v2.py
      compare_Nshape_Gcos_v3.py

    feature_versions/      # 全部为新增文件
      spec.py
      sources.py
      nmf_features.py
      grn_features.py
      distances.py
      fusion.py
      method_base.py
      export.py
      recipes.py

  configs/
    feature_versions/
      nkl_splitbeta_v1.yaml
      ncomp_gcos_v2.yaml
```

```
nshape_gcos_v3.yaml

tests/
  baseline/
    test_compare_N_kl_frozen.py
    test_feature_version_*.py
```

15.3 迁移顺序：不迁移旧代码，只新增平行实现

1. 生成并提交受保护文件哈希与旧方法小 fixture 回归快照；
2. 新建 pij/feature_versions/, 不在旧 _shared/features.py 继续追加实验逻辑；
3. 实现新版本自己的 FeatureVersionPijMethod, 直接遵守 PijMethod 接口；
4. 为每个算法语义增加一个薄 wrapper；
5. 最后只在 config.py 和 pij/registry.py 追加允许名称与映射；
6. 先运行旧 baseline 回归，再运行新入口 smoke test，二者均通过后才开始完整数据实验。

15.4 一个薄 PIJ 入口应该长什么样

```
from mignet_ce.pij.feature_versions.method_base import FeatureVersionPijMethod

class CompareNCompGCosV2PijMethod(FeatureVersionPijMethod):
    name = "compare_Ncomp_Gcos_v2"
    recipe_id = "ncomp_gcos_v2"
```

它不重复 NMF、JS、cosine、融合和 kernel 的实现，也不继承或覆写旧 CompareNKLPIjMethod。这样旧类的行为不会因为新版本扩展而发生变化。

15.5 配方文件示例

```
experiment_id: ncomp_gcos_v2
entry_method: compare_Ncomp_Gcos_v2
network_method: light_cci_grn

cci:
  extractor: directed_shared_core_nmf_all_layers
  rank: 5
  representation: separate_out_in_composition
  distance: jensen_shannon
  pseudocount: 1.0e-8
  seeds: [42]

grn:
  source: recompute_from_raw_expression_and_grn
  expression_transform: loglp_per_time_gene_minmax
```

```
roles: [regulator, target]
projection_dim: 64
role_combination: separate_costs
distance: cosine

fusion:
  normalization: robust_5_95_per_block
  weights:
    cci_out: 0.30
    cci_in: 0.30
    grn_reg: 0.20
    grn_tar: 0.20

kernel:
  type: exponential_row_normalized
  temperature: 1.0
```

15.6 版本命名规则

入口名只在数学语义变化时新增：

- compare_N_kl：冻结基线；
- compare_NG_kl_splitbeta_v1：同类信息源，N/G 独立温度；
- compare_Ncomp_Gcos_v2：NMF 组成距离 + GRN 两端 cosine；
- compare_Nshape_Gcos_v3：加入通信强度-形状分离；
- compare_Nresid_Gcos_consensus_v4：背景残差 + 成本共识。

rank、权重、种子和温度变化只产生新的 run ID 与 recipe hash，不复制完整 Python 方法。

15.7 结果目录与 run manifest

推荐结果路径：

```
output/
network=light_cci_grn/
experiment=njs_v2_1_strength_shape__8f31c2ab/
  metrics.csv
  run_manifest.json
  config_resolved.yaml
  diagnostics/
  logs/
```

run_manifest.json 至少记录：

- experiment ID、recipe 路径与 SHA256；
- Git commit；若没有 Git，则记录 code zip SHA256；

- 输入文件清单与关键输入 hash;
- Python、NumPy、SciPy、scikit-learn、PyTorch 版本;
- 全部随机种子、完整命令、开始/结束时间;
- 网络与 PIJ 方法名、resolved 配置;
- 是否 transductive、是否使用未来时间、是否使用 target 共同标准化。

16 实验路线：从最低成本定位到论文级创新

16.1 阶段 A：先解释当前结果

只运行心脏、当前 18 组：

1. exact baseline;
2. 新入口 V1: split-beta;
3. N-only;
4. G-only;
5. 0.75N+0.25G;
6. 新入口 V2 的消融：N composition + 合并 G cosine;
7. 新入口 V2 完整版：N composition + regulator/target 分离 cosine。

阶段 A 的输出目标不是“赢”，而是明确当前 13.5 负值来自哪一块。

16.2 阶段 B：确立 V2 主力表征

保持 GRN 方案不变，只比较：

- NMF 原生组成 + JS;
- NMF 原生组成 + Hellinger;
- 强度-形状分离;
- 所有层统一有向 shared-core NMF。

选出一个稳定的 N 路线后，再与 GRN 两端 cosine 融合。

16.3 阶段 C：提高稳定性和去背景能力

依次加入：

1. 3 个种子共识;
2. rank 小网格;

3. CCI 背景残差;
4. generalized-KL NMF;
5. regulon 活性。

每次只增加一个新变量，不做“残差 + 新 rank + 新 GRN + 新温度”同时改动。

16.4 阶段 D：跨器官和留一时间对验证

在 heart 选定候选后，至少在 brain/lungs 或另一批次验证。调参时不能反复读取同一 18 组 ΔEI ；应使用：

- 留一时间对：例如不使用所有指向 13.5 的结果选参数，再检验它们；
- 留一器官：在 heart 选结构，在另一个器官验证；
- 预先固定主要指标与次要诊断，避免事后挑指标。

16.5 阶段 E：只保留一个激进创新进入主论文

LR-program、扩散角色、层级一致性、多视图联合分解不应全部同时变成主方法。应根据 V2/V3 的结果选择一个最能解决现有失败模式、且有清晰生物学解释的方向。当前优先推荐顺序为：

LR-program > 层级一致性 > regulon 联合 > 扩散角色 > 黑箱对比学习。

17 验收标准：不能只看平均 ΔEI

17.1 一级指标

- mean/median ΔEI ;
- $\Delta EI > 0$ 比例;
- 各层级对、各时间对的稳定性;
- 当前四个负值是否改善，同时其他正值是否大幅回落。

17.2 二级信息指标

- lower/upper 原始 EI;
- 归一化 effectiveness;
- target 边缘熵 $H(J)$;
- 条件熵 $H(J | I)$;
- PIJ 平均行熵与有效支持数。

这样可以判断 ΔEI 增长是因为 **upper** 真正更可辨，还是 **lower** 被异常压低，或仅仅两边都被温度调尖。

17.3 三级表征质量

- NMF 重构误差与多种子稳定性；
- GRN 基因覆盖、regulon 覆盖、两端特征冗余；
- 特征与已知 domain/发育阶段的可视化，但不能把标签直接用于训练；
- 随机置换、单位标签置换和边权置换对照。

17.4 明确拒绝的“伪提升”

以下情况不能宣称特征更好：

1. 只降低 τ ，EI 上升但 PIJ 有效支持数大幅下降；
2. 在当前 18 组上直接最大化 ΔEI ，没有留出验证；
3. 改变上层与下层不同的参数，使比较失去公平性却未声明；
4. 只报告均值，掩盖某一时间点或层级系统性失败；
5. 使用全时间联合特征后仍把结果描述成严格的前向预测。

18 最终推荐：在旧入口零修改前提下实际先做哪三版

状态：只作为对照，不是待修改版本。保护其源文件哈希、默认参数、注册 key 和小型 fixture 结果。任何新实验都不得复用这个名字承载新数学逻辑。

目的：以最低风险判断 N 与 G 的 softmax 温度和融合权重是否造成 13.5 失稳。

实现：重用现有 NMF 与合并 GRN 信息源，但在新方法内部使用独立 β_N 、 β_G ；固定 $\tau = 1$ 做主比较；输出 N-only、G-only、分块成本和熵分解。

禁止：不得修改旧 `build_block_kl_cost()`，不得让旧 `compare_N_kl` 读取新参数。

目的：把当前信息源交给更符合语义的距离，是第一主线候选。

实现：所有非 gene 层按时间对重新做 directed shared-core NMF；发送和接收因子分别转为带伪计数的组成分布并计算 JS/Hellinger；从原始表达与 GRN 重新计算 regulator/target 两端，分别投影并用 cosine；三块成本独立归一后晚融合。

价值：不需要新数据，不修改网络基线，且消除了“随机坐标 softmax 成概率”和“两端相加抵消”两个主要语义问题。

目的：在 V2 基础上进一步分离通信总量和通信选择模式。
实现：NMF 之外显式计算发送/接收 log-strength；shape 使用组成距离，strength 使用稳健标量距离；所有层保持相同的 outgoing/incoming 语义；GRN 沿用 V2 已验证模块。
价值：若 13.5 的问题来自整体通信规模漂移，这一版本最有机会减少伪差异，同时保留真实角色变化。

只有 V2 或 V3 至少在多数时间对、多个层级上稳定优于基线，并通过种子稳定性检查后，才新增 compare_Nresid_Gcos_consensus_v4。V4 使用稀疏可计算的度背景残差和多种子成本共识，但不应与第一批三个入口同时实现，否则实验无法判断增益来自哪里。

19 附录 A：当前 exact baseline 参数表

参数/行为	当前值或实际语义
network method	light_cci_grn
PIJ method	compare_N_kl
organ	heart
time points	11.5、12.5、13.5、14.5
level pairs	spot→k150、spot→k40、k150→k40
NMF rank	5
NMF max iter	300
NMF seed	42
spot NMF	pairwise shared-core directed NMF, 特征 $[U V]$, 10 维
domain NMF	ordinary pairwise joint NMF, 特征 Z , 5 维
GRN top- K target	每个 regulator 50
GRN weight handling	取绝对值、重复边求和、行和归一化
GRN gate	double-end: $X \odot XW^T$ 与 $X \odot XW$
GRN projection dim	64
GRN projection seed	20260713

参数/行为	当前值或实际语义
GRN projection	基因名 + 角色 + 种子 SHA256 确定性高斯投影
GRN role combination	两端投影后直接相加
configured expression transform	loglp_minmax, 但当前 light_cci_grn 路径未执行
N/G softmax beta	同一个值 0.05, 来自 pij_entropy_epsilon
KL epsilon	10^{-12}
block weights	$w_N = w_G = 0.5$
block normalization	每块独立 5%–95% robust normalization
PIJ kernel	$\exp(-C/\tau)$ 后逐行归一化
PIJ temperature	$\tau = 1.0$
EI unit	bit
mean ΔEI	0.00529
median ΔEI	0.00630
positive ratio	$14/18 = 77.8\%$

20 附录 B：建议的 run manifest 示例

```
{
  "experiment_id": "njs_v2_0_comp_gcos",
  "network_method": "light_cci_grn",
  "pij_method": "compare_N_js_G_cos_v2",
  "recipe_sha256": "...",
  "code_commit": "...",
  "code_zip_sha256": "...",
  "input_manifest_sha256": "...",
  "organ": "heart",
  "time_points": ["11.5", "12.5", "13.5", "14.5"],
  "level_pairs": [
    "spot:seurat_k150",
    "spot:seurat_k40",
    "seurat_k150:seurat_k40"
  ],
  "random_seeds": {
    "nmf": [42],
    "grn_projection": 20260713
  },
  "transductive_pairwise_fit": true,
  "uses_target_for_scaling": true,
  "software_versions": {},
}
```

```
"command": "...",
"started_at": "...",
"finished_at": "...",
}
```

21 附录 C：主要符号总表

符号	含义
$A_t^{(\ell)}$	时间 t 、层级 ℓ 的 CCI 有向加权矩阵。
X_t	单位 $\times$ 基因表达矩阵。
W	top- K 、绝对值、行归一化后的 GRN 邻接矩阵。
S_t^{reg}	regulator 侧双端表达门控状态。
S_t^{tar}	target 侧双端表达门控状态。
$Q^{\text{reg}}, Q^{\text{tar}}$	两种角色的确定性基因哈希投影矩阵。
G_t	当前投影后的 GRN 单位特征。
U, V	有向 NMF 的发送与接收角色因子。
B	两时间共享的角色交互核心。
Z, H	普通 joint NMF 的单位因子与共享列基。
N_t	CCI-NMF 单位特征。
C_N, C_G	NMF 与 GRN 特征产生的跨时间成本。
$\hat{C}$	稳健归一化后的成本。
K	指数相似核。
P	行随机 PIJ 矩阵。
$H(J)$	target 边缘熵。
$H(J I)$	给定 source 干预后的条件熵。
EI	有效信息，等于 $H(J) - H(J I)$ 。
ΔEI	上层 EI 减下层 EI。

22 附录 D：新入口运行命令示例

旧基线命令保持原样：

```
python scripts/run_mignet_vertical.py \
--data-root /home/jovyan/public/datasets/Mouse-embryo/E15l_domain_factory \
```

```
--output-root /home/jovyan/work/2026\ Causality/output/baseline_compare_N_kl \  
--organs heart \  
--time-points 11.5 12.5 13.5 14.5 \  
--level-pairs spot:seurat_k150 spot:seurat_k40 seurat_k150:seurat_k40 \  
--network-method light_cci_grn \  
--pij-method compare_N_kl \  
--export-pij --export-pij-topk 0 --progress --max-workers 16
```

V1 仅替换新入口名：

```
python scripts/run_mignet_vertical.py \  
...same data/network arguments... \  
--network-method light_cci_grn \  
--pij-method compare_NG_kl_splitbeta_v1
```

V2 主线入口：

```
python scripts/run_mignet_vertical.py \  
...same data/network arguments... \  
--network-method light_cci_grn \  
--pij-method compare_Ncomp_Gcos_v2
```

不同方法必须使用不同 `--output-root` 或由现有输出层按 `pij_method` 自动隔离；禁止把新方法输出写入旧基线目录后覆盖文件。

23 附录 E：代码审查验收表

检查项	必须	验收方式
受保护文件 SHA256 未变化	是	自动测试读取快照并逐个比较。
旧 registry key 未变化	是	<code>get_pij_method("compare_N_kl")</code> 仍为旧类。
旧 fixture PIJ/EI 不变	是	<code>allclose</code> + 指标容差。
新入口各自独立输出	是	路径包含完整新方法名。
新入口不修改旧 payload	是	依赖图和代码审查。
reg/tar 两端从原始输入重算	V2+	artifact 中分别保存两端维度和摘要。
首轮 $\tau = 1$	是	recipe 与 manifest 检查。
同时报告 $H(J)$ 与 $H(J I)$	是	metrics/diagnostics 列存在。
多种子一致性	V4	成本矩阵相关性和方差报告。
内存受控	是	spot 层不构造额外 $n \times n$ dense 背景矩阵。

24 参考文献与方法来源

1. Lee, D. D., Seung, H. S. Algorithms for Non-negative Matrix Factorization. *Advances in Neural Information Processing Systems*, 2001.
2. Brunet, J.-P. et al. Metagenes and molecular pattern discovery using matrix factorization. *PNAS*, 2004, DOI: 10.1073/pnas.0308531101.
3. Aitchison, J. The Statistical Analysis of Compositional Data. *Journal of the Royal Statistical Society B*, 1982.
4. Endres, D. M., Schindelin, J. E. A new metric for probability distributions. *IEEE Transactions on Information Theory*, 2003, DOI: 10.1109/TIT.2003.813506.
5. Karrer, B., Newman, M. E. J. Stochastic blockmodels and community structure in networks. *Physical Review E*, 2011, DOI: 10.1103/PhysRevE.83.016107.
6. Aibar, S. et al. SCENIC: single-cell regulatory network inference and clustering. *Nature Methods*, 2017, DOI: 10.1038/nmeth.4463.
7. Argelaguet, R. et al. MOFA+: a statistical framework for comprehensive integration of multi-modal single-cell data. *Genome Biology*, 2020, DOI: 10.1186/s13059-020-02015-1.
8. Donnat, C. et al. Learning Structural Node Embeddings via Diffusion Wavelets. *KDD*, 2018.
9. Coifman, R. R., Maggioni, M. Diffusion wavelets. *Applied and Computational Harmonic Analysis*, 2006.
10. Jin, S. et al. Inference and analysis of cell-cell communication using CellChat. *Nature Communications*, 2021, DOI: 10.1038/s41467-021-21246-9.
11. Gauvin, L. et al. Detecting the community structure and activity patterns of temporal networks: a non-negative tensor factorization approach. *PLOS ONE*, 2014.

现有 `compare_N_kl` 必须冻结为不可变基线, 不能再承担任何新数学逻辑。第一批只新增三个入口: `split-beta` 诊断版、NMF 组成 + GRN 两端 `cosine` 主线版、通信强度-形状分离版。旧计算文件零修改; 只有 `config.py` 与 `registry.py` 做追加注册。所有新实现进入独立 `feature_versions` 包, 入口保持薄、`recipe` 与 `manifest` 可追溯, 并以文件哈希和数值回归共同保证旧结果不变。背景残差、共识 NMF、LR-program 与层级一致性在主线通过后再逐层加入。